

PRODUCT & TECHNICAL SPECIFICATION

AFORCE OS

The Real-Time Human Performance Operating System

PAUSE → HYDRATE → LOCK IN → PERFORM

BRAND SYSTEM V2.1.0

COMPILED JUNE 23, 2026

SOURCE: AFORCE OS CODEBASE

CONFIDENTIAL
DO NOT DISTRIBUTE

Contents

15 SECTIONS

01	Product Overview
02	Product Locks (non-negotiable design rules)
03	Technical Architecture
04	App Surfaces — Screens & Navigation
05	Feature Flags & Phased Exposure
06	Hydration Scoring Engine
07	Performance-State Color System
08	Subscription Tiers
09	Product Store (Physical SKUs)
10	Data Model (PostgreSQL / Drizzle)
11	API Surface (`api-server`)
12	Integrations
13	Localization
14	Publishing & Distribution
15	Design System (Brand v2.1.0)

1. Product Overview

AForce OS is a real-time human-performance operating system delivered as a React Native / Expo mobile app backed by an Express 5 + PostgreSQL API server. Its core job is **hydration intelligence**: it turns logged fluid intake, biometrics, and environment into a single live performance score, then coaches the user with an AI voice engine. Around that core it layers recovery, social accountability, competition, and an integrated store/subscription business.

Positioning mantra: *Pause → Hydrate → Lock In → Perform.* **Tagline:** *Performance Is Non-Negotiable.*

2. Product Locks (non-negotiable design rules)

These constrain every feature and cannot be silently overridden.

- Water-First Command System.** Recommendation order is **Water → Command → Optional support → Score update**. Default coach copy must begin with `HYDRATE NOW` / `Start with water`. Products never lead; behavior first, product second.
- Score Protection.** Only *completed* actions change the score. Recommendations, scans (HydroScan is advisory), and product selection never increase score. Every Home read-out is a read-only projection of already-completed behavior.
- Language / Localization Lock.** Launch languages: English, Spanish, French, German, Portuguese, Italian. No country-specific prioritization; architecture stays modular so hidden locales can ship later behind flags.
- Engine / UI Governance.** The engine may grow smarter; navigation may not grow. Feature flags control exposure. Principle: **Build 100% · Show 10% · Unlock over time**. One engine, multiple experiences.
- Product Positioning.** Decision order: Context → Recovery → Behavior → Learning → Optional support. Products support behavior; they never drive it.

3. Technical Architecture

Layer	Technology
Monorepo	pnpm workspaces
Mobile	React Native / Expo SDK 54, Expo Router 6, Reanimated 3, Gesture Handler
Mobile state	React Context + <code>useReducer</code> (slice-based store)
Data fetching	<code>@tanstack/react-query</code> with Orval-generated hooks
Backend	Node.js 24, Express 5, PostgreSQL, Drizzle ORM, Zod
API contract	OpenAPI-first; Orval codegen → React Query hooks + Zod schemas
Real-time	Shared HTTP + WebSocket server; REST mutations broadcast to clients
i18n	i18next
Auth	Clerk (<code>@clerk/expo</code> mobile, <code>@clerk/express</code> server)
Payments	Stripe + <code>stripe-replit-sync</code> (webhooks mirrored to Postgres)
Voice	ElevenLabs TTS (Expo Speech fallback)
Environment	OpenWeather API (server-proxied, cached)

Folder model (mobile): `app/` (routes/layouts), `components/`, `services/` (business logic), `store/` (reducer slices), `utils/` (pure helpers), `featureFlags/`, `theme/` (brand system), `design/` (Figma tokens), `data/` (SKUs, plans, templates), `types/`.

4. App Surfaces — Screens & Navigation

Navigation is **locked to 5 visible tabs**; additional surfaces exist as stacked or hidden routes (engines built, surfaces revealed over time).

Visible tabs: Home · Hydration (Journal) · Protocol · Community · Profile.

All routes (`app/`): - Tabs: `index` (Home), `protocol`, `scan` (HydroScan), `journal`, `competition`, `social`, `sleep`, `profile`, `social-legacy` (*several hidden via `href:null`*). - Auth: `(auth)/sign-in`, `(auth)/sign-up` (Clerk). - Hidden / staged: `(hidden)/cruise/*` (journey · port · excursion · recovery), `circles/[id] · manage`.

`shared` , `heat/guardian` , `ring/session` , `subscription/manage` , `legal/privacy · terms · health-disclaimer` , `clutch` (team command grid), `phantom` (wearable status), `territory` (performance map), `urine-check` , `onboarding` , `weekly-report` .

Home composition (current, post-revert orb-focused layout): readiness eyebrow → status headline → Status Pulse Orb → status label → live status line → consequence line → status-based primary CTA → quick-action grid → Hydration Status card → flag-gated zones (render nothing when off) → pinned "AFORCE" brand footer.

5. Feature Flags & Phased Exposure

Phase 1 (Core) is on by default; Phase 2/3 sub-features shipoff and light up over time. Verified defaults from `featureFlags/flags.ts` :

On by default (`true`): `city_competition_enabled` , `state_competition_enabled` , `team_competition_enabled` , `global_leaderboard_enabled` , `cruise_mode_enabled` , `sleep_mode_enabled` , plus spec flags `spec_activation` , `spec_social` , `spec_coachV2` , `spec_recoveryCircle` , `spec_notifications` , `spec_orb` , `spec_timelineLock` , `spec_hydroJournal` , `spec_hydroScan` , `spec_profileSource` , `spec_sharedContextLayer` , `spec_uiFreeze` .

Off by default (`false`): `clutch_access_enabled` (+ `heat/inventory/clip`), `guardian_intelligence_enabled` (+ `body_map/alerts`), `phantom_wearable_enabled` , `ring_enabled` , `kids_world_enabled` , `demo_mode_enabled` , `metabolic_readiness_enabled` (+ `glucose`), `performance_age_enabled` , `voice_checkin_enabled` , `intent_capture_enabled` , `performance_statements_enabled` , `offline_intake_outbox_enabled` , all `cruise_*` sub-features, `voice_status_module_visible` , and gated locales `spec_language_ar/zh/ja/ko/hi` .

6. Hydration Scoring Engine

A single 0–100 score drives every coach surface in lock-step.

- **Event-driven scoring.** Each logged intake event has defined point values, **absorption caps** (per rolling time window), and **time-release curves** so a large gulp is absorbed gradually rather than all at once.
- **Continuous depletion.** A pure, dependency-free helper computes score-points-per-minute decay from physiological standards, so the score falls between logs.
- **Daily water target** is derived per user from body weight, activity, and climate (heat/humidity from the weather proxy), not a fixed number.
- **Score Protection** is enforced at the engine boundary: scans and recommendations are advisory and never mutate score; only completed intake/recovery behavior does. Scoring runs **on-device**; the server persists client-submitted snapshots rather than acting as the authority.
- **Modifiers:** Social/Night Mode (alcohol) raises decay; Heat Guard and low-score states apply small product-specific recovery bonuses.

(Exact point/cap constants live in `utils/scoringEngine.ts` , `services/sweatRateEngine.ts` , and the depletion helpers — parameterized in code, intentionally not frozen here.)

7. Performance-State Color System

Five score bands (single source of truth: `theme/statusColor.ts` , brand v2.1.0). Color is a **signal only** — borders, dots, glows, accents, CTA tint — never a fill.

Band	Range	Calm hex	Meaning
OPTIMAL	85–100	<code>#1FA35A</code> (Soursop green)	Peak — soft wide glow
STABLE	70–84	<code>#3DBE7A</code> (green, lighter)	Holding — subtle glow
DECLINING	50–69	<code>#FFDE00</code> (amber)	Slipping — minimal glow
RISK	30–49	<code>#FF8C1A</code> (orange)	Act now — medium glow
CRITICAL	0–29	<code>#FF2800</code> (signal red)	Depleted — tight intense glow

A **Pressure Mode** variant deepens saturation, raises glow alpha, and speeds the pulse for the same band.

8. Subscription Tiers

Verified monthly prices from `data/subscriptionPlans.ts` .

Consumer (5 tiers):

Plan	Price	Unlocks
Core	\$9.99/mo	Hydration Control Center, Status Pulse + score, basic protocol, basic AI, logging, smart reminders
Recovery+	\$9.99/mo	Adds post-session Recovery Mode
AForce Athlete	\$19.99/mo	Enhanced AI, personalized protocol, advanced recovery, 90-day trends, competition + city/state/team leaderboards, premium notifications, Metabolic Readiness
Performance Bundle	\$59.99/mo	Flagship: Athlete tier + monthly product drop (1 canister or 2 stick packs) at member pricing, priority AI/insights
AForce Elite	\$99/mo	Guardian Mode (individual), premium analytics, full monthly product bundle, early access, concierge support

Team (Core): Team Core Starter \$49/mo (≤25 seats) · Team Core Growth \$99/mo (≤50) · Team Core Pro \$149/mo (≤100) — roster-aware, group reporting, admin console.

Enterprise — Clutch (live team command): Clutch Starter \$1,000/mo · Clutch Pro \$2,500/mo · Clutch Elite \$5,000/mo.

Enterprise — Guardian (injury-risk protection): Guardian Core \$5,000/mo + \$7,500 setup (6-mo min) · Guardian Elite \$8,000/mo + \$12,500 setup (12-mo min).

Entitlement is gated client-side via `useEntitlement.ts`; Stripe is the source of truth, mirrored to Postgres.

9. Product Store (Physical SKUs)

Catalog model: one SKU per (format × flavor), plus per-format bundles (`data/pricing.ts` , `data/products.ts`). Subscriber/member pricing is lower than list. Pricing/shipping/tax are computed server-side.

- **Formats:** Stick packs (12 ct) · RTD cans (12 pk) · Canisters (30 servings).
- **Flavors:** Berry · Watermelon · Soursop.
- **Bundles:** flavor-agnostic multi-packs (flavor split chosen at checkout).
- Per-serving sodium and serving sizes are encoded per SKU for the sweat-rate math.

10. Data Model (PostgreSQL / Drizzle)

Table	Key columns
<code>aforce_user_state</code>	<code>user_id</code> (PK), <code>units_consumed_today</code> , <code>oz_target</code> , <code>symptom_state</code> , <code>biometrics</code> (JSONB), <code>social_mode</code> (JSONB), <code>intake_events</code> (JSONB)
<code>aforce_intake_logs</code>	<code>id</code> , <code>user_id</code> , <code>fluid_type</code> , <code>oz_amount</code> , <code>score_before</code> , <code>score_after</code>
<code>aforce_score_snapshots</code>	<code>id</code> , <code>user_id</code> , <code>score</code> , <code>level</code> , <code>recovery_score</code> , <code>captured_at</code>
<code>aforce_users</code>	<code>id</code> , <code>stripe_customer_id</code> , <code>plan_id</code> , <code>subscription_status</code> , <code>referral_code</code>
<code>aforce_circle_users</code>	<code>owner_user_id</code> , <code>member_user_id</code> , <code>status</code>
<code>aforce_analytics_events</code>	<code>event_id</code> , <code>analytics_id</code> , <code>event_type</code> , <code>payload</code>

11. API Surface (`api-server`)

Express 5 with Zod input/output validation generated from the OpenAPI spec.

Method · Path	Purpose
GET /api/aforce/state	Full UserState snapshot
POST /api/aforce/intake	Log fluid consumption
POST /api/aforce/signals	Update active symptoms (mild/moderate/severe)
GET /api/aforce/journal/timeline	Interleaved snapshots + intake history
POST /api/aforce/social/activate	Start a Social-Mode drinking session
POST /api/aforce/social/cruise	Start 24h Enterprise Cruise Mode
GET /api/aforce/weather	OpenWeather proxy (tempC, humidity) by lat/lon
POST /api/voice/tts	ElevenLabs text-to-speech proxy
POST /api/checkout/session	Stripe Checkout for subscriptions
POST /api/stripe/webhook	Stripe events → updates <code>plan_id</code>

Hardening: `SELECT ... FOR UPDATE` to serialize concurrent user actions; rate limiting on public + authenticated endpoints; in-memory TTL cache on the weather proxy; structured request/non-request logging.

12. Integrations

- **Auth — Clerk.** Custom email/password + Google SSO. JWT via `Authorization: Bearer` (REST) and `?token=` (WebSocket). Token bridging into the OpenAPI client; auth-gated routes on both mobile and server.
- **Real-time — WebSocket** mounted on the shared server (`/api/aforce/ws`) with periodic heartbeats; REST mutations are broadcast to connected clients.
- **Payments — Stripe** via `stripe-replit-sync` ; the webhook secret is pulled from the managed Replit Stripe connector (not an env var). Webhook events mirror subscription state to Postgres.
- **Environment — OpenWeather**, server-proxied with a ~10-minute TTL cache (`OPENWEATHER_API_KEY`).
- **Voice — ElevenLabs** (`eleven_turbo_v2_5`); static phrases cached. Expo Speech is the fallback.

13. Localization

Launch set (6, in the selector): English · Spanish · French · German · Portuguese · Italian. Additional locales (Arabic, Chinese, Japanese, Korean, Hindi) are wired as resources but **gated behind flags** and excluded from the language selector. Architecture is modular so new languages add without a rebuild. (Number/date localization is a known gap — formatting is currently device-locale bound.)

14. Publishing & Distribution

- **Build service:** EAS Build — `development` , `preview` , `production` profiles in `eas.json` .
- **Bundle IDs:** iOS `com.aforce.os` , Android `com.aforce.os` .
- **Branding assets:** icon / splash / adaptive / favicon all render the **N-N "Non-Negotiable" monogram** (Bone `#F5F0E8` N's, Signal Red `#C1281B` center bar, Cinematic Black `#0D0D0D`), generated from a font-free vector SVG.
- **Build:** `pnpm eas:build:ios` · `:android` · `:all` · `:preview` (from `artifacts/aforce-os/`).
- **Submit:** `pnpm eas:submit:ios` (needs ASC App ID + Apple Team ID via the `eas-configure-submit` helper) · `pnpm eas:submit:android` (needs a Google Play service-account JSON, never committed).
- **Store screenshots** must be captured from a real device/simulator build — the web preview can't produce submission-grade assets.

15. Design System (Brand v2.1.0)

- **Canvas:** near-black `#0D0D0D` . Content floats; soft radial glows, never hard box shadows. Generous spacing.
- **Accents:** Signal Red `#C1281B` used sparingly (thin lines, eyebrows, active states, CTAs); Soursop green `#1FA35A` for positive status; Berry blue `#1E5BFF` for secondary data. Pure black `#000000` reserved for scrims/shadows/inverse text.
- **Type (three roles):** Archivo Black (display) · IBM Plex Mono (eyebrows / metrics) · Inter (body).
- **Tokens:** exported as `design/aforce-tokens.json` (Tokens Studio) + human-readable `design/aforce-design-tokens.md` . Served at `GET /api/design-tokens` and `GET /api/design-guide` .

Source of truth: the AForce OS codebase. Locked product decisions are maintained in `replit.md`; this spec mirrors them and adds the concrete, code-verified inventory.